МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №5. Хеширование

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Изучить метод хеширования на примере организации таблиц идентификаторов в процессе трансляции программ. Получить представление о преимуществах и недостатках, присущих различным методам организации таблиц символов (идентификаторов).

ПОСТАНОВКА ЗАДАЧИ

Для выполнения лабораторной работы требуется написать программу, которая получает на входе набор идентификаторов, организует таблицу по заданному методу и позволяет осуществить **многократный** поиск идентификатора в этой таблице. Список идентификаторов считать заданным в виде текстового файла. Длина идентификаторов ограничена 32 символами.

Вариант №5.

Требуется разработать программу, реализующую комбинированный способ организации таблицы идентификаторов.

- **Метод хеширования**: деление;
- **Способ разрешения коллизий**: открытая адресация (квадратичные пробы).

ХОД РАБОТЫ

В ходе лабораторной работы была разработана программа, предназначенная для организации хеш-таблицы с использованием метода открытой адресации.

Была реализована структура **HashCell**, представляющая собой ячейку таблицы (Рисунок 1). Ячейка хранит значение и идентификатор состояния (***пустая, занятая, свободная***), что необходимо для алгоритма открытой адресации.

```
/// @brief Состояния таблицы
enum CellState {
    EMPTY, // Ячейка пуста и никогда не использовалась
    OCCUPIED, // Ячейка занята элементом
    DELETED, // Ячейка была занята, но элемент удалён
};

/// @brief Ячейка хэш-таблицы
/// @tparam T тип данных
template <typename T>
struct HashCell {
    T value; // Значение/ключ
    CellState state; // Состояние ячейки

    // Конструктор по умолчанию (пустая ячейка)
    HashCell() : state(EMPTY), value(T()) {}
};
```

Рисунок 1 — Структура HashCell

На основе **HashCell** был реализован класс **HashTable**, представляющий собой хеш-таблицу с квадратичным пробированием (Рисунок 2).

```
/// @brief Хеш-таблица с квадратичным пробированием
/// @tparam T тип данных
template <typename T>
class HashTable {
private:
    vector<HashCell<T>> table; // Массив ячеек таблицы
    int size; // Общая ёмкость таблицы (размер массива)
    int cells_count; // Количество занятых элементов

    /// @brief Преобразование строки в число
    /// @param key входящая строка
    /// @return строку как вещественное число
    unsigned long long stringToLong(const string& key) const {
        unsigned long long sum = 0;
        for (char c : key) { // Проходим по каждому символу строки
            sum += static_cast<unsigned char>(c);
        }
        return sum;
    }

    /// @brief Хеш-функция методом деления
    /// @param key значение для определения индекса
    /// @return Индекс элемента
    int hash_function(const string& key) const {
        unsigned long long numeric_key = stringToLong(key); // Вычисляем числовое значение ключа
        return static_cast<int>(numeric_key % size); // Вычисляем индекс: остаток от деления на размер таблицы
    }
```

Рисунок 2 — Класс HashTable

Приватные атрибуты данного класса:

- динамический массив для хранения ячеек;
- текущий размер таблицы;
- количество занятых элементов;
- вспомогательный метод **stringToLong** — преобразует строку в вещественное число.

Метод **hash_function** — основная хеш-функция, работающая по принципу деления: индекс элемента вычисляется как остаток от деления на общий размер таблицы.

Ниже представлены публичные методы класса **HashTable**, необходимые для корректной реализации хеш-таблицы.

Очистка таблицы (Рисунок 3): приводит таблицу в начальное состояние, очищая все ячейки и изменяя их флаги состояния на ***EMPTY***.

```
/// @brief Очистка таблицы. Сбрасывает до изначального состояния
void clear() {
    for (auto& entry : table) {
        entry.state = EMPTY;
        entry.value = T();
    }
    cells_count = 0;
}
```

*Рисунок 3 — метод **clear***

Вставка элемента (Рисунок 4): реализация добавления элемента в таблицу с разрешением коллизии методом квадратичного пробирования. Функция вычисляет начальный индекс элемента **default_index** и шаг пробирования **i**. Затем запускается цикл поиска подходящей ячейки. Если ячейка помечена как удалённая, алгоритм запоминает её и переходит к следующему индексу по формуле: **index = (default_index + i * i) % size**. После завершения цикла поиска элемент вставляется в удалённую ячейку (если таковая была найдена) или в найденную пустую. В противном случае вставка не происходит (таблица заполнена).

```
/// @brief Вставка элемента
/// @param key значение для вставки
/// @return результат вставки как `boolean`
bool insert(const T& key) {
    if (cells_count >= size) {
        cout << "Error: Table is full!" << endl;
        return false;
    }

    int default_index = hash_function(key); // Вычисляем начальный индекс

    int i = 0; // Счётчик шагов пробирования
    int index = default_index; // Текущий индекс элемента
    int firstDeletedIndex = -1; // Индекс первой найденной ячейки со статусом DELETED

    while (table[index].state != EMPTY) { // Ищем место для вставки
        if (table[index].state == OCCUPIED && table[index].value == key) {
            return false; // Дубликат найден, вставка не требуется
        }

        if (table[index].state == DELETED && firstDeletedIndex == -1) {
            firstDeletedIndex = index; // Запоминаем индекс
        }
        i++;
        index = (default_index + i * i) % size; // Квадратичное пробирование

        if (i >= size) { // Защита от бесконечного цикла
            break;
        }
    }

    if (firstDeletedIndex != -1) { // Была найдена удалённая ячейка
        table[firstDeletedIndex].value = key;
        table[firstDeletedIndex].state = OCCUPIED;
    }
    else if (table[index].state == EMPTY) { // Была найдена пустая ячейка
        table[index].value = key;
        table[index].state = OCCUPIED;
    }
    else {
        cout << "Error: Cannot find space!" << endl;
        return false;
    }
    cells_count++;
    return true;
}
```

Рисунок 4 — метод ***insert***

Удаление элемента (Рисунок 5): алгоритм поиска ячейки схож с методом **insert**. Функция ищет ячейку с нужным значением, перебирая индексы методом квадратичного пробирования. Если ячейка была найдена, она помечается как удалённая, а размер массива уменьшается.

```
/// @brief Удаление элемента
/// @param key значение для удаления
/// @return результат удаления как `boolean`
bool remove(const T& key) {
    int default_index = hash_function(key); // Вычисляем начальный индекс
    int i = 0; // Счётчик шагов
    int index = default_index; // Текущий индекс для поиска

    while (table[index].state != EMPTY) { // Найдена пустая ячейка с таким значением => конец цепочки поиска
        if (table[index].state == OCCUPIED && table[index].value == key) { // Найдена нужная ячейка
            table[index].state = DELETED;
            cells_count--;
            return true;
        }
        i++;
        index = (default_index + i * i) % size; // Квадратичное пробирование

        if (i >= size) { // Защита от бесконечного цикла
            break;
        }
    }
    return false;
}
```

*Рисунок 5 — метод **remove***

Поиск элемента (Рисунок 6): функция проходится по всем ячейкам таблицы, перебирая индексы методом квадратичного пробирования. При этом ведётся подсчёт количества сравнений и коллизий при выполнении алгоритма, информация о которых выводится после завершения цикла поиска.

```
/// @brief Поиск элемента
/// @param key значение для удаления
void search(const T& key) {
    int default_index = hash_function(key); // Вычисляем начальный индекс
    int i = 0; // Счётчик шагов
    int index = default_index; // Текущий индекс для поиска

    int comparisons = 0; // Счётчик сравнений
    int collisions = 0; // Счётчик коллизий (занятые, но не подходящие ячейки)

    cout << "\n--- Search Results ---" << endl;
    cout << "Key: " << key << endl;
    cout << "hash_function: " << default_index << endl;

    while (table[index].state != EMPTY) { // Цикл поиска элемента
        comparisons++;

        if (table[index].state == OCCUPIED) {
            if (table[index].value == key) {
                cout << "STATUS: FOUND at index " << index << endl;
                break;
            }
            else {
                collisions++;
            }
        }

        i++;
        index = (default_index + i * i) % size; // Квадратичное пробирование

        if (i >= size) { // Защита от бесконечного цикла
            break;
        }
    }

    if (table[index].state == EMPTY || i >= size) {
        cout << "STATUS: NOT FOUND" << endl;
    }
    cout << "Comparisons: " << comparisons << ", Collisions: " << collisions << endl;
    cout << "------------------------" << endl;
}
```

*Рисунок 6 — метод **search***

Статистическая сводка (Рисунок 7): функция проходится по всем ячейкам таблицы, перебирая индексы методом квадратичного пробирования. Алгоритм имитирует процесс поиска для каждого элемента, параллельно подсчитывая общее количество сравнений и коллизий. После выполнения основного цикла выводится статистика по таблице: размер, количество элементов, процент загруженности, среднее количество сравнений и коллизий для всех элементов.

```
/// @brief Вывод статистики. Считает, сколько нужно сделать шагов чтобы найти элемент
void printAverageStats() {
    if (cells_count == 0) {
        cout << "Table is empty, stats unavailable." << endl;
        return;
    }

    long long totalComparisons = 0; // Всего сравнений
    long long totalCollisions = 0;  // Всего коллизий

    for (int i = 0; i < size; ++i) { // Проход по всей таблице
        if (table[i].state == OCCUPIED) {
            T key = table[i].value;
            int default_index = hash_function(key); // начальный индекс
            int steps_count = 0; // Счётчик шагов для элемента
            int current_index = default_index; // Текущий индекс элемента


            while (true) { // Имитируем процесс поиска элемента
                totalComparisons++;

                if (table[current_index].state == OCCUPIED && table[current_index].value == key) {
                    totalCollisions += steps_count; // Ячейка найдена
                    break;
                }
                steps_count++;
                current_index = (default_index + steps_count * steps_count) % size; // Квадратичное пробирование

                // Защита от бесконечного цикла
                if (steps_count >= size) {
                    break;
                }
            }
        }
    }

    // Выводим итоговую статистику
    cout << "\n=== STATISTICS FOR REPORT ===" << endl;
    cout << "Size: " << size << ", Elements: " << cells_count << endl;
    cout << "Load factor: " << fixed << setprecision(2) << (double)cells_count / size * 100 << "%" << endl;
    cout << "Avg comparisons: " << (double)totalComparisons / cells_count << endl;
    cout << "Avg collisions:  " << (double)totalCollisions / cells_count << endl;
    cout << "============================" << endl;
}
```

Рисунок 7 — метод ***printAverageStats***

Помимо основных методов управления таблицей класс **HashTree** включает в себя:

- метод загрузки данных в файл (Рисунок 8);
- метод выгрузки данных из файла (Рисунок 9);
- метод вывода всей таблицы (Рисунок 10).

```
/// @brief Выгрузка данных в файл
/// @param filename имя файла
void saveToFile(const string& filename) {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "Error: Cannot create file " << filename << endl;
        return;
    }

    for (int i = 0; i < size; ++i) {
        if (table[i].state == OCCUPIED) { // Берём только занятые ячейки
            file << table[i].value << endl;
        }
    }
    file.close();
    cout << "Data saved to file." << endl;
}
```

*Рисунок 8 — метод **saveToFile***

```
/// @brief Загрузка данных из файла
/// @param filename имя файла
void loadFromFile(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Error: Cannot open file " << filename << endl;
        return;
    }

    clear();

    string word; // Переменная для хранения слова из файла

    while (file >> word) {
        if (word.length() > MAX_STRING_LENGTH) {
            word = word.substr(0, MAX_STRING_LENGTH);
        }
        insert(word);
    }
    file.close();
    cout << "Data loaded from file." << endl;
}
```

*Рисунок 9 — метод **loadFromFile***

```
/// @brief Вывод таблицы в консоль
void printTable() {
    cout << "\n--- Hash Table Contents ---" << endl;
    cout << "Index | State | Value" << endl; // Индекс | Состояние | Значение
    cout << "------|---------|---------------" << endl;

    for (int i = 0; i < size; ++i) {
        // Выводим ячейки, которые OCCUPIED
        if (table[i].state == OCCUPIED) {
            cout << i << " | OK | " << table[i].value << endl;
        }
        // Выводим ячейки, которые DELETED
        else if (table[i].state == DELETED) {
            cout << i << " | DEL | " << endl;
        }
    }
    cout << "-----------------------------" << endl;
}
```

*Рисунок 10 — метод **printTable***

Для управления программой было реализовано консольное меню, поддерживающее вызов указанных ранее методов для работы с хеш-таблицей (Рисунки 11-13).

```
int main() {
    HashTable<string> ht(37);

    int choice;
    string buffer;

    while (true) {
        cout << "\nLABORATORY WORK #5 (Variant 5)\n";
        cout << "Method: Division, Collision Resolution: Open Addressing (Quadratic Probing)\n";
        cout << "1. Load from file\n";
        cout << "2. Save to file\n";
        cout << "3. Search element\n";
        cout << "4. Add element\n";
        cout << "5. Remove element\n";
        cout << "6. Show table\n";
        cout << "7. General Statistics (for report)\n";
        cout << "0. Exit\n";

        cout << "Your choice: ";

        if (!(cin >> choice)) { // Защита от ввода букв вместо цифр
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (choice) {
        case 1: // Загрузка из файла
            cout << "Enter filename: ";
            cin >> buffer;
            ht.loadFromFile(buffer);
            break;
        case 2: // Выгрузка в файл
            cout << "Enter filename to save: ";
            cin >> buffer;
            ht.saveToFile(buffer);
            break;
```

Рисунок 11 — консольное меню — часть 1

```
        case 3: // Поиск элемента
            cout << "Enter word to search: ";
            cin >> buffer;
            ht.search(buffer);
            break;
        case 4: // Добавление элемиента
            cout << "Enter word to add: ";
            cin >> buffer;
            if (ht.insert(buffer))
                cout << "Successfully added.\n";
            else
                cout << "Not added (duplicate or table full).\n";
            break;
        case 5: // Удаление элемента
            cout << "Enter word to remove: ";
            cin >> buffer;
            if (ht.remove(buffer))
                cout << "Successfully removed.\n";
            else
                cout << "Element not found.\n";
            break;
        case 6: // Вывод таблицы
            ht.printTable();
            break;
        case 7: // Статистическая сводка
            ht.printAverageStats();
            break;
        case 0: // Выход
            cout << "Exit.\n";
            return 0;
        default:
            cout << "Invalid choice.\n";
        }
    }
```

Рисунок 12 — консольное меню — часть 2

```
LABORATORY WORK #5 (Variant 5)
Method: Division, Collision Resolution: Open Addressing (Quadratic Probing)
1. Load from file
2. Save to file
3. Search element
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice:
```

Рисунок 13 — интерфейс меню

Загрузим данные из заготовленного файла (Рисунки 14-15). Данные файла были распределены по хеш-таблице в соответствии с их индексами, вычисленными с помощью метода квадратичного пробирования.

```
1. Load from file
2. Save to file
3. Search element
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 1
Enter filename: input.txt
Data loaded from file.
```

Рисунок 14 — загрузка данных — часть 1

```
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 6

--- Hash Table Contents ---
Index | State | Value
------|---------|---------------
0 | OK | four
7 | OK | six
11 | OK | eight
13 | OK | two
18 | OK | three
19 | OK | five
20 | OK | nine
26 | OK | one
27 | OK | seven
31 | OK | ten
-----------------------------
```

Рисунок 15 — загрузка данных — часть 2

Выполним поиск слова «***nine***» (Рисунок 16). Функция успешно нашла требуемый элемент, а также вывела начальное значение индекса, вычисленное хеш-функцией, количество сравнений и количество коллизий.

```
3. Search element
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 3
Enter word to search: nine

--- Search Results ---
Key: nine
hash_function: 19
STATUS: FOUND at index 20
Comparisons: 2, Collisions: 1
--------------------------
```

Рисунок 16 — поиск элемента

Добавим новый элемент — слово «***twelve***» (Рисунки 17-19). Добавление прошло успешно, новый слово корректно находится с помощью функции поиска, а также отображается при выводе всей таблицы.

```
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 4
Enter word to add: twelve
Successfully added.
```

Рисунок 17 — добавление элемента — часть 1

```
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 6

--- Hash Table Contents ---
Index | State | Value
------|--------|---------------
0 | OK | four
7 | OK | six
11 | OK | eight
13 | OK | two
18 | OK | three
19 | OK | five
20 | OK | nine
26 | OK | one
27 | OK | seven
31 | OK | ten
34 | OK | twelve
-----------------------------
```

Рисунок 18 — добавление элемента — часть 2

```
3. Search element
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 3
Enter word to search: twelve

--- Search Results ---
Key: twelve
hash_function: 34
STATUS: FOUND at index 34
Comparisons: 1, Collisions: 0
------------------------
```

Рисунок 19 — добавление элемента — часть 3

Удалим элемент «***nine***» из таблицы (Рисунки 20-22). Удаление проходит успешно, соответствующая ячейка изменяет свой флаг, а поиск удалённого элемента не даёт результатов.

```
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 5
Enter word to remove: nine
Successfully removed.
```

Рисунок 20 — удаление элемента — часть 1

```
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 6

--- Hash Table Contents ---
Index | State | Value
------|---------|----------------
0 | OK | four
7 | OK | six
11 | OK | eight
13 | OK | two
18 | OK | three
19 | OK | five
20 | DEL |
26 | OK | one
27 | OK | seven
31 | OK | ten
34 | OK | twelve
-----------------------------
```

Рисунок 21 — удаление элемента — часть 2

```
3. Search element
4. Add element
5. Remove element
6. Show table
7. General Statistics (for report)
0. Exit
Your choice: 3
Enter word to search: nine

--- Search Results ---
Key: nine
hash_function: 19
STATUS: NOT FOUND
Comparisons: 2, Collisions: 1
-------------------------
```

Рисунок 22 — удаление элемента — часть 3

Выведем статистическую сводку по таблице (Рисунки 23-24). Функция корректно выводит общий размер таблицы, количество элементов в ней, процент загруженности и среднее количество сравнений. После полного заполнения таблицы элементами данные изменились. В частности, при поиске одного элемента в среднем стало происходить большее число сравнений и коллизий.

```
7. General Statistics (for report)
0. Exit
Your choice: 7

=== STATISTICS FOR REPORT ===
Size: 37, Elements: 10
Load factor: 27.03%
Avg comparisons: 1.00
Avg collisions:  0.00
============================
```

Рисунок 23 — статистическая сводка — часть 1

```
7. General Statistics (for report)
0. Exit
Your choice: 7

=== STATISTICS FOR REPORT ===
Size: 37, Elements: 37
Load factor: 100.00%
Avg comparisons: 3.27
Avg collisions:  2.27
============================
```

Рисунок 24 — статистическая сводка — часть 2

ВЫВОДЫ

В ходе выполнения лабораторной работы был изучен принцип организации таблиц с использованием метода хеширования. Для реализации хеш-таблицы в качестве метода хеширования использовался **метод деления**, а в качестве способа разрешения коллизий была использована **квадратичная пробация**.

Так, было установлено, что при заполнении хеш-таблицы элементами среднее число сравнений составило **3.27**, при этом значение при загрузке на треть — **1.00**. Увеличение количества сравнений незначительно, что говорит о высокой эффективности использования метода хеширования для организации структур данных, в частности использование **метода деления** и **квадратичной пробации**.